

Serial Protocol Decode — Keithley DMM6500, DAQ6510, DMM7510

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 1.05 — beta** · MIT licence (see [LICENSE](#)) · [user manual](#) · [measured reference](#)

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the firmware, so the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

What to know before trusting it

Tested on a DMM6500 and nothing else. Every measured number in these documents came off that one instrument. See [Which instruments](#).

Flow control is verified electrically but never closed as a loop. One credit pulse per armed capture, idle-low at **+0.20 V**, **+4.45 V high**, **5.72 µs wide**, **436 ns rise**, with nothing that waits for it ever on the other end. The width is **not settable** — `trigger.extout.pulsewidth` is nil on 1.7.17a — so a receiver needs an edge-triggered input rather than a polling loop. On this bench the generator's burst machinery accepts the arbs and does not act on the pulse; **Triggering, in both directions** in [REFERENCE.md](#) has it gate by gate.

Automatic rate detection has two known failures, both rare, both fixed by typing the rate in. A short pattern repeated over and over — `00 / FF / 55 / AA`, walking bits, a fixed string on a loop — can measure at a small multiple of its true rate: **24 of 6 714** bench points across four sweeps, **0.36 %**, every one a repeating pattern at a non-standard rate. Offline the split is **0 of 6 314 standard-rate points** against 24 of 6 027 non-standard; on the instrument a **LIN-style break field**, not valid 8N1 at all, measured at three times its true rate on standard rates, silently. Separately, a logic low near **−2 V** can misdetect, while **−0.1**, **−0.5**, **−1.0** and **−3.0 V** measure clean. Both are characterised under **Known failures of automatic rate detection** in the [manual](#).

Nothing stops a long job early. A touch press cannot be delivered while a script runs, and the front-panel TRIGGER key does **not** deliver `trigger.EVENT_DISPLAY` during a panel-initiated run. A recording runs to its stated size. That key does still *gate* an armed capture under `Options ▶ Trigger = Trigger key`, verified by `tools/bench_trigkey.py --quiet-line`.

How much it captures

You need	One press gets you
a look at the line	~ 240 bytes , on screen
a burst to a file	8 192 or 32 768 bytes — recorded, decoded and filed, no stepping
more than that	flow control turns the window into a chunk size: credit, record, decode, credit, with no interaction per window

Flow control is bounded at **32 windows or 20 minutes** per press. Mode and Capture resume the same conversation in a new file, the sender still waiting on its credit. Per-mode ceilings and the arithmetic are in [REFERENCE.md](#).

Endurance, measured — and reproduced

Two independent 17-hour soaks, each **6 laps of the full 1 683-point matrix, 10 098 capture-and-decode cycles on one power cycle**, with **no instrument event logged, no buffer or display object leaked and no error left behind**. Different cabling, and a change to the app's state handling between them:

	duration	failures of 10 098	drift
first	17.14 h	318	−0.9 %
second	17.17 h	313	−1.5 %

Within 0.2 % on duration and 1.6 % on failure count across 20 196 cycles.

Lap time is the leak indicator — a leaked display object, an unreturned buffer handle or a fragmenting allocator wedges the instrument around hour six without failing a single point. It *fell* in both runs: the second's laps ran 10 567, 10 219, 10 254, 10 282, 10 173 and 10 299 s, each within ± 1 % of its counterpart in the first.

Comparing which fixed-stimulus cells failed, **a lap of the second run differs from its counterpart in the first by less than two laps of the first differ from each other** — 11–16 cells against 21–32.

3.1 % of those points fail, against a matrix built to attack the decoder: LIN traffic presented to a UART decoder, single-byte and walking-bit patterns, sinusoidal drift, swings and offsets swept to the edge of range. Not comparable to the rate-detection figure above. Two properties of that 3.1 %:

- **It concentrates.** Four vectors — `v94` , `v63` , `v90` , `v71` — account for **76 %** of all failures; `v63` is a LIN frame whose 13-bit break field is not valid 8N1.
- **It is mostly intermittent.** The 318 failures are **162 distinct cells**, of which **88 failed in exactly one lap of the six** and **6 in all six**.

A third, 7-hour soak covers the **recording** path, which neither long soak exercises: **81 laps, 3 726 cycles and 8 one-press recordings, zero failures**, every recording ending on its byte ceiling.

Not covered by any soak: many power cycles rather than one, and front-panel interaction.

Which instruments

The app needs a digitizer reachable as `dmm.digitize` , the **touchscreen app API** (`display.create` and friends), and **Lua 5.0.2**. Every digitizer below runs at **1 MS/s**.

DMM6500	Tested , firmware 1.7.17a. 16-bit digitizer
DAQ6510	Should run unmodified, on the strongest grounds of the three: it shares the UI board and the acquisition board with the DMM6500, only the channel-board plugin differing. Untested
DMM7510	Should run unmodified: near-identical front panel, 18-bit digitizer, better acquisition boards. Untested
2461 SMU	Would need a port — dual 18-bit digitizers , reached as <code>smu.</code> rather than <code>dmm.</code>

2470 SMU	No. "Digitized measurements are not a feature on the 2470" — its reference manual, rev D, October 2024
2450 / 2460 SMU	Almost certainly not, by absence rather than denial: neither claims a digitizer, and the 2450 reference documents <code>smu.measure.*</code> with no digitize function

The timing figures transfer; the analog ones may not. Every rate figure in REFERENCE descends from the sample rate and a buffer that accepts ~100 000 readings/s. Resolution is the one axis this app is indifferent to, thresholding each sample into a one or a zero. Measured *through the acquisition board* — the tolerance envelope, the verified level and offset limits, the −2 V band — is the part to re-measure: the DAQ6510 shares that board, so a deviation there is a real finding, while the DMM7510's differs and those figures may move in either direction.

The riskiest part to port is not the 14 `dmm.*` calls but `serial_ui.tsp`: 1 300 lines of pixel-positioned display code against baked-in panel constants — a content area **49 px** below the panel top, a **31-character** title limit, **122 display objects** from a pool the firmware does not fully reclaim.

The `.tspa` header decides where it installs, independently of whether the code would run. The build declares `$Product: DMM6500, DAQ6510, DMM7510`. SMUs are absent: `$Product:` means "can run the script without errors", and an SMU would install and then fail at the first capture. `DMM7510` is not in the value set Keithley's app-header spec publishes, though Keithley ship TSP apps of their own declaring it — an install failure is the symptom if a firmware validates the field strictly, and this field is the first thing to revert.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order. `docs/BENCH.md` describes the harness in full.

```
python3 tools/release_sweep.py --offline # ~1 min, no instruments
python3 tools/bench_smoke.py # 11 min, both instruments
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96
```

The smoke gate covers the whole rate range in 11 minutes. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise, so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family faces both formats and both content classes. Then all 45 button presses. 86 cells in 9.5 min; presses in 1.9.

No version is tagged without at least 8 hours of soak, and the last two took 17. A soak reports a **failure rate per test point**; a sweep reports one pass or one failure. A lap is 1 683 cells and about 2.9 hours, so 8 hours is the least that separates "fails every lap" from "failed once", and 17 gives six laps.

Code changes do not get pushed without passing the smoke gate. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/`; a `pre-push` hook recomputes it and refuses if either tree has moved since.

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Documentation-only pushes are unaffected; neither tree is hashed. When the bench is unavailable — mid-soak, say — `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason.

Every lap draws its waveform order, its non-standard rates and the wait before each capture from the lap number, so `--iteration N` rebuilds any lap exactly without running the ones before it, and a failure replays offline in seconds. `docs/BENCH.md` has the commands.

```
python3 tools/release_sweep.py
```

That is the gate. It runs every check in one go and writes an auditable record to `out/release/<timestamp>/` — a `REPORT.md`, a `summary.json`, every stage's full output, every front-panel screenshot, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails.

It needs a freshly power-cycled DMM6500: `sdec.start()` builds the display objects and the app refuses a second build in one power cycle. It also needs the SDG2122X loaded with the stimulus waveforms (`bench_matrix.py --upload`), and only one client may hold the DMM's control socket at a time.

The offline half needs no instruments and runs in seconds:

```
python3 tools/release_sweep.py --offline
```

Stage	Checks
<code>lint</code> <code>parse</code>	Lua 5.0.2 incompatibilities; <code>luac -p</code> on every module
<code>vecrefs</code>	every waveform id named in <code>tools/</code> is in <code>MAP</code> or declared <code>RETIRED</code> — deleting from <code>MAP</code> alone leaves references that fail where they are used, not where they are declared
<code>soakrand</code>	<code>mt19937.py</code> and <code>mt19937.lua</code> produce one sequence, checked against CPython's <code>random</code> , plus the 5.0.2 scan deciding whether the module can load on the instrument
<code>unit</code>	1 063 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
<code>unit-cancel</code>	one-press recordings, both window sizes, and the flow-control loop with no interaction
<code>unit-frontrig</code>	TRIGGER-key and rear-BNC arming <i>through</i> <code>acquire()</code> — the routing that can drop to free-run while the status row still claims the key is the source
<code>unit-usblog</code>	USB log name allocation and the persistent index: exhaustion must refuse rather than loop and hang the panel
<code>unit-stream</code>	the streaming arm — both 4915 defences, one press per slice rather than per window
<code>unit-patterns</code>	byte-exactness on the hard payloads: edge-density extremes, walking bits and known random data, at the sample rates the panel actually picks
<code>unit-forcerate</code>	a forced rate the wire does not carry must refuse and name the rate it does carry, with both answers drivable unattended
<code>unit-judgev</code>	the harness's three verdicts — too short to read is inconclusive , not silently wrong; an alignment survives one bad byte; a loud vector may miss a bounded few. Each with the wrong capture that must still fail
<code>stress</code>	hostile signals: never silently wrong , never raises
<code>unit-analog</code>	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the window opens

Stage	Checks
unit-phasesweep	every vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among trustworthy calls
unit-seam	a capture whose arb loop seam lands late must still be judged, and the narrower trim still required
unit-sdguard	every route by which an out-of-spec waveform could reach the generator refuses it
unit-loremgate	the harness's own long-payload verdict: every clean run validated, flag count bounded
tolerance	recomputes the envelope table printed in the manual
package archive	rebuilds the <code>.tspa</code> , then builds both screens <i>from the archive</i> against a mock front end
manual	rebuilds every shipped PDF: <code>MANUAL.pdf</code> , <code>REFERENCE.pdf</code> , <code>README.pdf</code>
hw-matrix	through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, three logic swings, twelve DC offsets
hw-payloads	fourteen payloads covering every byte value 0–255, two of them also at 115 200 and 250 000
hw-odd-rates	nineteen non-standard baud rates — 900, 1500, 3600, 8123, 29127, 104857 ...
hw-panel	every button in every state, including a one-press recording. Six checks per press: the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, and the panel actually shows it — grabbed before and after each press and differenced by region
soakrand-dmm	the third leg: the instrument's own Lua 5.0.2 must produce the same words, floats, rejected draws and permutation
hw-plan	the seeded sweep on the bench: two waveforms across all 43 rates in a seeded order, with a seeded wait before every capture
hw-break	degenerate signals and contradictory settings — no signal, DC only, all- <code>0x00</code> / <code>0xFF</code> / <code>0x55</code> , a break, 60 mV of swing, 19 Vpp, rates past the ceiling, six wrong forced settings. A refusal with a reason passes; confident garbage does not

Layout

tsp/	the app. <code>serial_core</code> acquisition, <code>uart_decode</code> framing, <code>chunk_decode</code> resumable decode, <code>serial_ui</code> panel, <code>serial_app</code> orchestration
tools/	harnesses. <code>release_sweep.py</code> is the entry point; the rest are the authorities it calls
docs/	the manual, instrument references, panel mockups
notes/	bring-up log, findings, handoffs

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E verifying the stimulus is what it claims to be. `tools/instruments.py` holds the addresses and the hazards — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.